

Resolución de una Matriz A en C++

Generated by Doxygen 1.16.1

1 Resolución de una Matriz A en C++	1
1.1 Características	1
1.2 Estructura del proyecto	1
1.3 Documentación	1
1.4 Ejecución	2
1.5 Autora	2
1.6 Maestro	2
1.7 Materia y Programa Académico	2
1.8 Fecha	2
2 File Index	3
2.1 File List	3
3 File Documentation	5
3.1 main.cpp File Reference	5
3.1.1 Detailed Description	5
3.1.2 Function Documentation	6
3.1.2.1 main()	6
3.1.2.2 menu()	6
3.2 main.cpp	7
3.3 matriz.cpp File Reference	8
3.3.1 Detailed Description	8
3.3.2 Function Documentation	9
3.3.2.1 matrizInversa()	9
3.3.2.2 metodoGauss()	10
3.3.2.3 productoMatrices()	10
3.3.2.4 verMatriz()	11
3.3.2.5 verVector()	11
3.4 matriz.cpp	12
3.5 matriz.h File Reference	14
3.5.1 Detailed Description	14
3.5.2 Function Documentation	14
3.5.2.1 matrizInversa()	14
3.5.2.2 metodoGauss()	15
3.5.2.3 productoMatrices()	16
3.5.2.4 verMatriz()	17
3.5.2.5 verVector()	17
3.6 matriz.h	18
3.7 README.md File Reference	18
Index	19

Chapter 1

Resolución de una Matriz A en C++

Implementación de un matriz A y un vector B con funciones de visualización de contenido, así como su resolución mediante los métodos de Gauss y de Gauss-Jordan con la inversa de la matriz. Tanto la matriz como el vector representan la presión en un tanque de almacenamiento.

1.1 Características

Visualización de...

- La matriz A y el vector B.
- La resolución del sistema de ecuaciones lineales mediante el método de Gauss.
- La matriz inversa de A y la resolución del sistema mediante el método de Gauss-Jordan.

1.2 Estructura del proyecto

- doc/
 - html/
 - latex/
- [matriz.h](#)
- [matriz.cpp](#)

1.3 Documentación

Toda la documentación ha sido generada con Doxygen.

Para visualizar: Diríjase a `doc/html/index.html` Posteriormente, abra con su navegador de preferencia.

Para visualizar todos los comentarios: Vaya a "Class Members", después, seleccione "All".

O abra el archivo PDF "refman.pdf".

1.4 Ejecución

- Usando comandos: Abra su terminal, y ejecute el siguiente código...

```
g++ main.cpp matriz.cpp -o matriz ./matriz
```

1.5 Autora

Camila Andrea Jaimes Martinez.

1.6 Maestro

Miguel Angel Reyes Gerónimo.

1.7 Materia y Programa Académico

- Álgebra lineal.
- Ingeniería en Software y Minería de Datos (1er año).

1.8 Fecha

Viernes 24 de Abril, 2026.

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

main.cpp	El programa completo para la manipulación de la matriz A y el vector B	5
matriz.cpp	Implementación de los constructores para resolver el sistema de ecuaciones lineales representado en la matriz A y el vector B	8
matriz.h	Define los constructores para resolver sistemas de ecuaciones lineales mediante matrices . .	14

Chapter 3

File Documentation

3.1 main.cpp File Reference

El programa completo para la manipulación de la matriz A y el vector B.

```
#include <iostream>
#include "matriz.h"
```

Functions

- int [menu](#) ()
Menú del programa que maneja el input (aportación) del usuario.
- int [main](#) ()

3.1.1 Detailed Description

El programa completo para la manipulación de la matriz A y el vector B.

Las opciones son las siguientes:

- Opción: 1 Visualizar la matriz A y el vector B.
- Opción 2: Visualizar la resolución del sistema de ecuaciones lineales mediante el método de Gauss.
- Opción 3: Visualizar la matriz inversa de A y la resolución del sistema mediante el método de Gauss-Jordan.
- Opción 4: Salir del programa.

Definition in file [main.cpp](#).

3.1.2 Function Documentation

3.1.2.1 main()

```
int main ()
```

Definition at line 18 of file [main.cpp](#).

```
00019 {
00020     vector<vector<double>> A = {
00021         {4, -1, -1, 0, 0},
00022         {-1, 3, 0, -1, 0},
00023         {-1, 0, 3, 0, -1},
00024         {0, -1, -1, 4, -1},
00025         {0, 0, -1, -1, 3}
00026     };
00027
00028     vector<double> b = {50, 0, 10, 0, 20};
00029
00030     while (true)
00031     {
00032         switch (menu())
00033         {
00034             case 1:
00035             {
00036                 cout << "\n--- Matriz A ---\n";
00037                 verMatriz(A);
00038
00039                 cout << "\n--- Vector B ---\n";
00040                 verVector(b);
00041                 break;
00042             }
00043             case 2:
00044             {
00045                 vector<double> sol = metodoGauss(A, b);
00046
00047                 cout << "\n--- Valores finales ---\n";
00048                 for (int i = 0; i < sol.size(); i++)
00049                 {
00050                     cout << "P" << i+1 << " = " << sol[i] << endl;
00051                 }
00052
00053                 break;
00054             }
00055             case 3:
00056             {
00057                 auto inv = matrizInversa(A, b);
00058
00059                 cout << "\n--- Valores finales ---\n";
00060                 for (int i = 0; i < inv.size(); i++)
00061                 {
00062                     cout << "P" << i+1 << " = " << inv[i] << endl;
00063                 }
00064
00065                 break;
00066             }
00067             case 4:
00068             {
00069                 cout << "\nSaliendo del programa...";
00070                 exit(0);
00071                 break;
00072             }
00073         }
00074     }
00075 }
```

3.1.2.2 menu()

```
int menu ()
```

Menú del programa que maneja el input (aportación) del usuario.

Devuelve una opción a la función [main\(\)](#).

Definition at line 85 of file [main.cpp](#).

```

00086 {
00087     int opcion;
00088     do
00089     {
00090         cout << "\n\n\t--- MENU! ---";
00091         cout << "\n\n1) Ver la matriz A y el vector B.\n";
00092         cout << "2) Resolver la matriz A mediante el método de Gauss.\n";
00093         cout << "3) Obtener la matriz inversa de A y resolución mediante el método de Gauss-Jordan.\n";
00094         cout << "4) Salir del programa.\n";
00095         cout << "\n--> Seleccione una opción: ";
00096         cin >> opcion;
00097     } while (opcion < 1 || opcion > 4);
00098     return opcion;
00099 }

```

3.2 main.cpp

[Go to the documentation of this file.](#)

```

00001 #include <iostream>
00002 #include "matriz.h"
00003
00014
00015 int menu();
00016 using namespace std;
00017
00018 int main()
00019 {
00020     vector<vector<double>> A = {
00021         {4, -1, -1, 0, 0},
00022         {-1, 3, 0, -1, 0},
00023         {-1, 0, 3, 0, -1},
00024         {0, -1, -1, 4, -1},
00025         {0, 0, -1, -1, 3}
00026     };
00027
00028     vector<double> b = {50, 0, 10, 0, 20};
00029
00030     while (true)
00031     {
00032         switch (menu())
00033         {
00034             case 1:
00035             {
00036                 cout << "\n--- Matriz A ---\n";
00037                 verMatriz(A);
00038
00039                 cout << "\n--- Vector B ---\n";
00040                 verVector(b);
00041                 break;
00042             }
00043             case 2:
00044             {
00045                 vector<double> sol = metodoGauss(A, b);
00046
00047                 cout << "\n--- Valores finales ---\n";
00048                 for (int i = 0; i < sol.size(); i++)
00049                 {
00050                     cout << "P" << i+1 << " = " << sol[i] << endl;
00051                 }
00052
00053                 break;
00054             }
00055             case 3:
00056             {
00057                 auto inv = matrizInversa(A, b);
00058
00059                 cout << "\n--- Valores finales ---\n";
00060                 for (int i = 0; i < inv.size(); i++)
00061                 {
00062                     cout << "P" << i+1 << " = " << inv[i] << endl;
00063                 }
00064
00065                 break;
00066             }
00067             case 4:
00068             {
00069                 cout << "\nSaliendo del programa...\n";
00070                 exit(0);
00071                 break;
00072             }
00073         }
00074     }
00075 }

```

```

00074     }
00075 }
00076
00077 /*****
00078
00084
00085 int menu()
00086 {
00087     int opcion;
00088     do
00089     {
00090         cout << "\n\n\t--- MENU! ---";
00091         cout << "\n\n1) Ver la matriz A y el vector B.\n";
00092         cout << "2) Resolver la matriz A mediante el método de Gauss.\n";
00093         cout << "3) Obtener la matriz inversa de A y resolución mediante el método de Gauss-Jordan.\n";
00094         cout << "4) Salir del programa.\n";
00095         cout << "\n--> Seleccione una opción: ";
00096         cin >> opcion;
00097     } while (opcion < 1 || opcion > 4);
00098     return opcion;
00099 }

```

3.3 matriz.cpp File Reference

Implementación de los constructores para resolver el sistema de ecuaciones lineales representado en la matriz A y el vector B.

```

#include <iostream>
#include <iomanip>
#include "matriz.h"

```

Functions

- void [verMatriz](#) (const vector< vector< double > > &A)
Imprime el contenido de la matriz A.
- void [verVector](#) (const vector< double > &b)
Imprime el contenido del vector B, el cual corresponde a los resultados de cada ecuación del sistema lineal.
- vector< double > [metodoGauss](#) (vector< vector< double > > A, vector< double > b)
Resuelve la matriz A mediante el método de eliminación de Gauss, devolviendo el valor de las incógnitas del sistema.
- vector< double > [matrizInversa](#) (vector< vector< double > > A, const vector< double > &b)
Obtiene la matriz inversa de A utilizando el método de eliminación de Gauss-Jordan.
- vector< double > [productoMatrices](#) (const vector< vector< double > > &A, const vector< double > &b)
Multiplca la matriz A por el vector B, lo cual es útil para calcular el producto de la matriz inversa de A por el vector B.

3.3.1 Detailed Description

Implementación de los constructores para resolver el sistema de ecuaciones lineales representado en la matriz A y el vector B.

Definition in file [matriz.cpp](#).

3.3.2 Function Documentation

3.3.2.1 matrizInversa()

```
vector< double > matrizInversa (
    vector< vector< double > > A,
    const vector< double > & b)
```

Obtiene la matriz inversa de A utilizando el método de eliminación de Gauss-Jordan.

Se utiliza la matriz identidad para transformar la matriz A en una matriz aumentada, posteriormente, se normaliza cada fila (el pivote se vuelve 1) y elimina los coeficientes debajo del pivote (los coeficientes debajo del pivote se vuelven 0), a tal modo que la matriz A se transforme en una matriz identidad.

La matriz identidad, una vez transformada, se convierte en la matriz inversa de A, la cual es necesaria para encontrar las incógnitas del sistema de ecuaciones lineales al multiplicarla por el vector B.

Parameters

<i>a</i>	Matriz de coeficientes del sistema de ecuaciones lineales.
<i>b</i>	Vector de resultados del sistema de ecuaciones lineales.

Definition at line 120 of file [matriz.cpp](#).

```
00121 {
00122     int n = A.size();
00123     vector<vector<double>> I(n, vector<double>(n, 0));
00124
00125     for (int i = 0; i < n; i++)
00126         I[i][i] = 1;
00127
00128     cout << "\n--- MATRIZ INVERSA DE A ---\n";
00129
00130     for (int i = 0; i < n; i++) {
00131         double pivot = A[i][i];
00132
00133         for (int j = 0; j < n; j++) {
00134             A[i][j] /= pivot;
00135             I[i][j] /= pivot;
00136         }
00137
00138         cout << "\nNormalizando fila " << i+1 << ":\n";
00139         verMatriz(A);
00140
00141         for (int k = 0; k < n; k++)
00142         {
00143             if (k != i)
00144             {
00145                 double factor = A[k][i];
00146                 for (int j = 0; j < n; j++)
00147                 {
00148                     A[k][j] -= factor * A[i][j];
00149                     I[k][j] -= factor * I[i][j];
00150                 }
00151             }
00152         }
00153
00154         cout << "\nEliminando columna " << i+1 << ":\n";
00155         verMatriz(A);
00156     }
00157
00158     cout << "\n--- MATRIZ INVERSA DE A ---\n";
00159     verMatriz(I);
00160
00161     vector<double> x = productoMatrices(I, b);
00162
00163     cout << "\n--- RESULTADO DEL MÉTODO DE GAUSS-JORDAN ---\n";
00164     for (int i = 0; i < n; i++)
00165     {
00166         cout << "P" << i+1 << " = " << x[i] << endl;
00167     }
00168
00169     return x;
00170 }
```

3.3.2.2 metodoGauss()

```
vector< double > metodoGauss (
    vector< vector< double > > A,
    vector< double > b)
```

Resuelve la matriz A mediante el método de eliminación de Gauss, devolviendo el valor de las incógnitas del sistema.

Transforma la matriz A, primero, en una matriz aumentada de A que incluye el vector B, posteriormente, se transforma en una matriz triangular superior, normalizando cada fila (el pivote se vuelve 1) y eliminando los coeficientes debajo del pivote (los coeficientes debajo del pivote se vuelven 0).

Para encontrar el valor de las incógnitas, se sustituye a partir de la última fila hacia arriba.

Parameters

<i>A</i>	Matriz de coeficientes del sistema de ecuaciones lineales.
<i>b</i>	Vector de resultados del sistema de ecuaciones lineales.

Definition at line 60 of file [matriz.cpp](#).

```
00061 {
00062     int n = A.size();
00063
00064     cout << "\n--- MÉTODO DE GAUSS ---\n";
00065
00066     for (int i = 0; i < n; i++)
00067     {
00068         double pivot = A[i][i];
00069
00070         for (int j = i; j < n; j++)
00071             A[i][j] /= pivot;
00072         b[i] /= pivot;
00073
00074         cout << "\nPaso " << i+1 << " (Normalizar fila " << i+1 << "):\n";
00075         verMatriz(A);
00076         verVector(b);
00077
00078         for (int k = i+1; k < n; k++)
00079         {
00080             double factor = A[k][i];
00081             for (int j = i; j < n; j++)
00082                 A[k][j] -= factor * A[i][j];
00083             b[k] -= factor * b[i];
00084         }
00085
00086         cout << "\nEliminación en columna " << i+1 << ":\n";
00087         verMatriz(A);
00088         verVector(b);
00089     }
00090
00091     vector<double> x(n);
00092
00093     for (int i = n-1; i >= 0; i--) {
00094         x[i] = b[i];
00095         for (int j = i+1; j < n; j++)
00096             x[i] -= A[i][j] * x[j];
00097     }
00098
00099     cout << "\n--- RESULTADO DEL MÉTODO DE GAUSS ---\n";
00100     verVector(x);
00101
00102     return x;
00103 }
```

3.3.2.3 productoMatrices()

```
vector< double > productoMatrices (
    const vector< vector< double > > & A,
    const vector< double > & b)
```

Multiplica la matriz A por el vector B, lo cual es útil para calcular el producto de la matriz inversa de A por el vector B.

Se multiplica cada fila de la matriz A por el vector B, sumando los productos para encontrar el valor de cada incógnita del sistema de ecuaciones lineales.

Parameters

<i>a</i>	Matriz de coeficientes del sistema de ecuaciones lineales.
<i>b</i>	Vector de resultados de sistemas de ecuaciones lineales.

Definition at line 182 of file [matriz.cpp](#).

```
00183 {
00184     int n = A.size();
00185     vector<double> result(n, 0);
00186
00187     for (int i = 0; i < n; i++)
00188     {
00189         for (int j = 0; j < n; j++)
00190         {
00191             result[i] += A[i][j] * b[j];
00192         }
00193     }
00194
00195     return result;
00196 }
```

3.3.2.4 verMatriz()

```
void verMatriz (
    const vector< vector< double > > & A)
```

Imprime el contenido de la matriz A.

Se itera sobre cada fila de la matriz A, imprimiendo los coeficientes respectivos.

Parameters

<i>A</i>	Matriz de coeficientes del sistema de ecuaciones lineales.
----------	--

Definition at line 23 of file [matriz.cpp](#).

```
00024 {
00025     for (auto& row : A) {
00026         for (auto val : row)
00027             cout << setw(8) << val << " ";
00028         cout << endl;
00029     }
00030 }
```

3.3.2.5 verVector()

```
void verVector (
    const vector< double > & b)
```

Imprime el contenido del vector B, el cual corresponde a los resultados de cada ecuación del sistema lineal.

Se itera sobre cada elemento del vector B, imprimiendo los resultados respectivos.

Parameters

<i>b</i>	Vector de resultados del sistema de ecuaciones lineales.
----------	--

Definition at line 40 of file [matriz.cpp](#).

```
00041 {
00042     for (auto val : b)
00043         cout << setw(8) << val << endl;
00044 }
```

3.4 matriz.cpp

[Go to the documentation of this file.](#)

```
00001 #include <iostream>
00002 #include <iomanip>
00003 #include "matriz.h"
00004
00005 using namespace std;
00006
00012 using namespace std;
00014
00015 /*****
00016
00022
00023 void verMatriz(const vector<vector<double>>& A)
00024 {
00025     for (auto& row : A) {
00026         for (auto val : row)
00027             cout << setw(8) << val << " ";
00028         cout << endl;
00029     }
00030 }
00031
00032 /*****
00033
00039
00040 void verVector(const vector<double>& b)
00041 {
00042     for (auto val : b)
00043         cout << setw(8) << val << endl;
00044 }
00045
00046 /*****
00047
00059
00060 vector<double> metodoGauss(vector<vector<double>> A, vector<double> b)
00061 {
00062     int n = A.size();
00063
00064     cout << "\n--- MÉTODO DE GAUSS ---\n";
00065
00066     for (int i = 0; i < n; i++)
00067     {
00068         double pivot = A[i][i];
00069
00070         for (int j = i; j < n; j++)
00071             A[i][j] /= pivot;
00072         b[i] /= pivot;
00073
00074         cout << "\nPaso " << i+1 << " (Normalizar fila " << i+1 << "):\n";
00075         verMatriz(A);
00076         verVector(b);
00077
00078         for (int k = i+1; k < n; k++)
00079         {
00080             double factor = A[k][i];
00081             for (int j = i; j < n; j++)
00082                 A[k][j] -= factor * A[i][j];
00083             b[k] -= factor * b[i];
00084         }
00085
00086         cout << "\nEliminación en columna " << i+1 << ":\n";
00087         verMatriz(A);
00088         verVector(b);
00089     }
00090 }
```



```

00091     vector<double> x(n);
00092
00093     for (int i = n-1; i >= 0; i--) {
00094         x[i] = b[i];
00095         for (int j = i+1; j < n; j++)
00096             x[i] -= A[i][j] * x[j];
00097     }
00098
00099     cout << "\n--- RESULTADO DEL MÉTODO DE GAUSS ---\n";
00100     verVector(x);
00101
00102     return x;
00103 }
00104
00105 /*****
00106
00107
00108
00109
00110
00111
00112
00113
00114
00115
00116
00117
00118
00119
00120 vector<double> matrizInversa(vector<vector<double>> A, const vector<double>& b)
00121 {
00122     int n = A.size();
00123     vector<vector<double>> I(n, vector<double>(n, 0));
00124
00125     for (int i = 0; i < n; i++)
00126         I[i][i] = 1;
00127
00128     cout << "\n--- MATRIZ INVERSA DE A ---\n";
00129
00130     for (int i = 0; i < n; i++) {
00131         double pivot = A[i][i];
00132
00133         for (int j = 0; j < n; j++) {
00134             A[i][j] /= pivot;
00135             I[i][j] /= pivot;
00136         }
00137
00138         cout << "\nNormalizando fila " << i+1 << ":\n";
00139         verMatriz(A);
00140
00141         for (int k = 0; k < n; k++)
00142         {
00143             if (k != i)
00144             {
00145                 double factor = A[k][i];
00146                 for (int j = 0; j < n; j++)
00147                 {
00148                     A[k][j] -= factor * A[i][j];
00149                     I[k][j] -= factor * I[i][j];
00150                 }
00151             }
00152         }
00153
00154         cout << "\nEliminando columna " << i+1 << ":\n";
00155         verMatriz(A);
00156     }
00157
00158     cout << "\n--- MATRIZ INVERSA DE A ---\n";
00159     verMatriz(I);
00160
00161     vector<double> x = productoMatrices(I, b);
00162
00163     cout << "\n--- RESULTADO DEL MÉTODO DE GAUSS-JORDAN ---\n";
00164     for (int i = 0; i < n; i++)
00165     {
00166         cout << "P" << i+1 << " = " << x[i] << endl;
00167     }
00168     return x;
00169 }
00170
00171 /*****
00172
00173
00174
00175
00176
00177
00178
00179
00180
00181
00182 vector<double> productoMatrices(const vector<vector<double>>& A, const vector<double>& b)
00183 {
00184     int n = A.size();
00185     vector<double> result(n, 0);
00186
00187     for (int i = 0; i < n; i++)
00188     {
00189         for (int j = 0; j < n; j++)
00190         {
00191             result[i] += A[i][j] * b[j];
00192         }
00193     }
00194
00195     return result;
00196 }

```

3.5 matriz.h File Reference

Define los constructores para resolver sistemas de ecuaciones lineales mediante matrices.

```
#include <vector>
```

Functions

- void [verMatriz](#) (const vector< vector< double > > &A)
Imprime el contenido de la matriz A.
- void [verVector](#) (const vector< double > &b)
Imprime el contenido del vector B, el cual corresponde a los resultados de cada ecuación del sistema lineal.
- vector< double > [metodoGauss](#) (vector< vector< double > > A, vector< double > b)
Resuelve la matriz A mediante el método de eliminación de Gauss, devolviendo el valor de las incógnitas del sistema.
- vector< double > [matrizInversa](#) (vector< vector< double > > A, const vector< double > &b)
Obtiene la matriz inversa de A utilizando el método de eliminación de Gauss-Jordan.
- vector< double > [productoMatrices](#) (const vector< vector< double > > &A, const vector< double > &b)
Multiplica la matriz A por el vector B, lo cual es útil para calcular el producto de la matriz inversa de A por el vector B.

3.5.1 Detailed Description

Define los constructores para resolver sistemas de ecuaciones lineales mediante matrices.

El presente archivo contiene los prototipos de las funciones necesarias para imprimir el contenido de la matriz A y el vector B del problema situado, así como para resolver la matriz A utilizando el método de eliminación de Gauss, obtener la matriz inversa de A, y multiplicar la matriz A por el vector B, con el propósito de encontrar el valor de las incógnitas del sistema de ecuaciones lineales.

Author

Camila A. J. M.

Definition in file [matriz.h](#).

3.5.2 Function Documentation

3.5.2.1 [matrizInversa\(\)](#)

```
vector< double > matrizInversa (  
    vector< vector< double > > A,  
    const vector< double > & b)
```

Obtiene la matriz inversa de A utilizando el método de eliminación de Gauss-Jordan.

Se utiliza la matriz identidad para transformar la matriz A en una matriz aumentada, posteriormente, se normaliza cada fila (el pivote se vuelve 1) y elimina los coeficientes debajo del pivote (los coeficientes debajo del pivote se vuelven 0), a tal modo que la matriz A se transforme en una matriz identidad.

La matriz identidad, una vez transformada, se convierte en la matriz inversa de A, la cual es necesaria para encontrar las incógnitas del sistema de ecuaciones lineales al multiplicarla por el vector B.

Parameters

<i>a</i>	Matriz de coeficientes del sistema de ecuaciones lineales.
<i>b</i>	Vector de resultados del sistema de ecuaciones lineales.

Definition at line 120 of file [matriz.cpp](#).

```

00121 {
00122     int n = A.size();
00123     vector<vector<double>> I(n, vector<double>(n, 0));
00124
00125     for (int i = 0; i < n; i++)
00126         I[i][i] = 1;
00127
00128     cout << "\n--- MATRIZ INVERSA DE A ---\n";
00129
00130     for (int i = 0; i < n; i++) {
00131         double pivot = A[i][i];
00132
00133         for (int j = 0; j < n; j++) {
00134             A[i][j] /= pivot;
00135             I[i][j] /= pivot;
00136         }
00137
00138         cout << "\nNormalizando fila " << i+1 << ":\n";
00139         verMatriz(A);
00140
00141         for (int k = 0; k < n; k++)
00142         {
00143             if (k != i)
00144             {
00145                 double factor = A[k][i];
00146                 for (int j = 0; j < n; j++)
00147                 {
00148                     A[k][j] -= factor * A[i][j];
00149                     I[k][j] -= factor * I[i][j];
00150                 }
00151             }
00152         }
00153
00154         cout << "\nEliminando columna " << i+1 << ":\n";
00155         verMatriz(A);
00156     }
00157
00158     cout << "\n--- MATRIZ INVERSA DE A ---\n";
00159     verMatriz(I);
00160
00161     vector<double> x = productoMatrices(I, b);
00162
00163     cout << "\n--- RESULTADO DEL MÉTODO DE GAUSS-JORDAN ---\n";
00164     for (int i = 0; i < n; i++)
00165     {
00166         cout << "P" << i+1 << " = " << x[i] << endl;
00167     }
00168
00169     return x;
00170 }

```

3.5.2.2 metodoGauss()

```

vector< double > metodoGauss (
    vector< vector< double > > A,
    vector< double > b)

```

Resuelve la matriz A mediante el método de eliminación de Gauss, devolviendo el valor de las incógnitas del sistema.

Transforma la matriz A, primero, en una matriz aumentada de A que incluye el vector B, posteriormente, se transforma en una matriz triangular superior, normalizando cada fila (el pivote se vuelve 1) y eliminando los coeficientes debajo del pivote (los coeficientes debajo del pivote se vuelven 0).

Para encontrar el valor de las incógnitas, se sustituye a partir de la última fila hacia arriba.

Parameters

<i>A</i>	Matriz de coeficientes del sistema de ecuaciones lineales.
<i>b</i>	Vector de resultados del sistema de ecuaciones lineales.

Definition at line 60 of file [matriz.cpp](#).

```

00061 {
00062     int n = A.size();
00063
00064     cout << "\n--- MÉTODO DE GAUSS ---\n";
00065
00066     for (int i = 0; i < n; i++)
00067     {
00068         double pivot = A[i][i];
00069
00070         for (int j = i; j < n; j++)
00071             A[i][j] /= pivot;
00072         b[i] /= pivot;
00073
00074         cout << "\nPaso " << i+1 << " (Normalizar fila " << i+1 << "):\n";
00075         verMatriz(A);
00076         verVector(b);
00077
00078         for (int k = i+1; k < n; k++)
00079         {
00080             double factor = A[k][i];
00081             for (int j = i; j < n; j++)
00082                 A[k][j] -= factor * A[i][j];
00083             b[k] -= factor * b[i];
00084         }
00085
00086         cout << "\nEliminación en columna " << i+1 << ":\n";
00087         verMatriz(A);
00088         verVector(b);
00089     }
00090
00091     vector<double> x(n);
00092
00093     for (int i = n-1; i >= 0; i--) {
00094         x[i] = b[i];
00095         for (int j = i+1; j < n; j++)
00096             x[i] -= A[i][j] * x[j];
00097     }
00098
00099     cout << "\n--- RESULTADO DEL MÉTODO DE GAUSS ---\n";
00100     verVector(x);
00101
00102     return x;
00103 }

```

3.5.2.3 productoMatrices()

```

vector< double > productoMatrices (
    const vector< vector< double > > & A,
    const vector< double > & b)

```

Multiplica la matriz A por el vector B, lo cual es útil para calcular el producto de la matriz inversa de A por el vector B.

Se multiplica cada fila de la matriz A por el vector B, sumando los productos para encontrar el valor de cada incógnita del sistema de ecuaciones lineales.

Parameters

<i>a</i>	Matriz de coeficientes del sistema de ecuaciones lineales.
<i>b</i>	Vector de resultados de sistemas de ecuaciones lineales.

Definition at line 182 of file [matriz.cpp](#).

```
00183 {
00184     int n = A.size();
00185     vector<double> result(n, 0);
00186
00187     for (int i = 0; i < n; i++)
00188     {
00189         for (int j = 0; j < n; j++)
00190         {
00191             result[i] += A[i][j] * b[j];
00192         }
00193     }
00194
00195     return result;
00196 }
```

3.5.2.4 verMatriz()

```
void verMatriz (
    const vector< vector< double > > & A)
```

Imprime el contenido de la matriz A.

Se itera sobre cada fila de la matriz A, imprimiendo los coeficientes respectivos.

Parameters

A	Matriz de coeficientes del sistema de ecuaciones lineales.
----------	--

Definition at line 23 of file [matriz.cpp](#).

```
00024 {
00025     for (auto& row : A) {
00026         for (auto val : row)
00027             cout << setw(8) << val << " ";
00028         cout << endl;
00029     }
00030 }
```

3.5.2.5 verVector()

```
void verVector (
    const vector< double > & b)
```

Imprime el contenido del vector B, el cual corresponde a los resultados de cada ecuación del sistema lineal.

Se itera sobre cada elemento del vector B, imprimiendo los resultados respectivos.

Parameters

b	Vector de resultados del sistema de ecuaciones lineales.
----------	--

Definition at line 40 of file [matriz.cpp](#).

```
00041 {
00042     for (auto val : b)
00043         cout << setw(8) << val << endl;
00044 }
```

3.6 matriz.h

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002 #include <vector>
00003
00013
00014 using namespace std;
00015
00016 void verMatriz(const vector<vector<double>>& A);
00017 void verVector(const vector<double>& b);
00018
00019 vector<double> metodoGauss(vector<vector<double>> A, vector<double> b);
00020 vector<double> matrizInversa(vector<vector<double>> A, const vector<double>& b);
00021 vector<double> productoMatrices(const vector<vector<double>>& A, const vector<double>& b);
```

3.7 README.md File Reference

Index

- main
 - main.cpp, [6](#)
- main.cpp, [5](#)
 - main, [6](#)
 - menu, [6](#)
- matriz.cpp, [8](#)
 - matrizInversa, [9](#)
 - metodoGauss, [9](#)
 - productoMatrices, [10](#)
 - verMatriz, [11](#)
 - verVector, [11](#)
- matriz.h, [14](#)
 - matrizInversa, [14](#)
 - metodoGauss, [15](#)
 - productoMatrices, [16](#)
 - verMatriz, [17](#)
 - verVector, [17](#)
- matrizInversa
 - matriz.cpp, [9](#)
 - matriz.h, [14](#)
- menu
 - main.cpp, [6](#)
- metodoGauss
 - matriz.cpp, [9](#)
 - matriz.h, [15](#)
- productoMatrices
 - matriz.cpp, [10](#)
 - matriz.h, [16](#)
- README.md, [18](#)
- Resolución de una Matriz A en C++, [1](#)
- verMatriz
 - matriz.cpp, [11](#)
 - matriz.h, [17](#)
- verVector
 - matriz.cpp, [11](#)
 - matriz.h, [17](#)